

Template System Acceptance Checklist

Date: 2026-02-15

Source Guide: [docs/development/template-system-comprehensive-guide.md](#)

A. Scope and Baseline Verification

- ☐ Changes are limited to template-system related files and docs.
- ☐ No unrelated architecture or design-system changes were introduced.
- ☐ Source-of-truth guide and specs were used for validation.

B. Functional Acceptance

AC-1: Template catalog is complete and loadable

- ☐ `config/templates.json` parses successfully.
- ☐ All built-in template IDs are unique.
- ☐ Each template has required metadata and at least one executable script definition.

AC-2: Core service contract is implemented end-to-end

- ☐ `ITemplateService` operations are implemented by `TemplateService`:
 - ☐ load/search/get
 - ☐ apply
 - ☐ validate
 - ☐ compatibility
 - ☐ import/export custom templates
 - ☐ history

AC-3: Template execution pipeline works correctly

- ☐ Variable resolution order is correct and deterministic.
- ☐ Preflight checks execute before scripts.
- ☐ Required preflight failure blocks execution with actionable message.
- ☐ Ordered script execution is respected.
- ☐ `ContinueOnError` behavior matches metadata.
- ☐ Progress events are emitted during execution.

AC-4: Security and safety constraints are enforced

- ☐ Absolute script paths are rejected.
- ☐ Path traversal attempts are blocked.
- ☐ Script path resolution is limited to allowed roots.
- ☐ Cancellation and timeout handling are functional.

AC-5: Desktop wizard integration is operational

- ☐ Template selection step appears in install workflow.

- ☐ Template apply step executes after install phase.
- ☐ User selections (template + variables) are applied correctly.
- ☐ Apply errors are surfaced without crashing the wizard.

AC-6: PowerShell command surface is consistent

- ☐ Module manifest exports template cmdlets.
- ☐ `Get-DistroNexusTemplate` supports ID/category filtering.
- ☐ `Apply-DistroNexusTemplate` applies templates to target instances.
- ☐ `Invoke-DistroNexusTemplateAutomation` supports full and selective modes.

AC-7: Automation runner classification and artifacts are correct

- ☐ Runner returns correct `Pass` / `Fail` / `Blocked` statuses.
- ☐ Capability-gated templates can be excluded or marked blocked with reason.
- ☐ CI guard behavior matches policy (`skip by default unless override`).
- ☐ Run artifacts are generated:
 - ☐ XML report
 - ☐ run manifest JSON
 - ☐ markdown summary

AC-8: Built-in template coverage is aligned with current catalog

- ☐ Built-in templates are present across categories currently implemented:
 - ☐ `Development`
 - ☐ `Platform`
 - ☐ `CloudNative`
 - ☐ `Database`
 - ☐ `DataAndAI`
- ☐ Category gap status for `DevOps` target is explicitly documented.

AC-9: History and observability are available

- ☐ Application history persists to `%APPDATA%\DistroNexus\template-application-history.json`.
- ☐ Template execution logs/progress can be inspected during or after apply.

C. Non-Functional Acceptance

- ☐ Scripts are idempotent or include clear rerun safety constraints.
- ☐ Distro compatibility assumptions are declared and validated.
- ☐ Error messages are actionable for operators and contributors.
- ☐ Documentation reflects current implementation behavior.

D. Evidence Checklist

- ☐ Attach catalog validation output.
- ☐ Attach selective-run evidence (single template).
- ☐ Attach selective-run evidence (multiple templates).
- ☐ Attach full-run evidence (or reason not executed).

- ☐ Attach generated artifact paths under [docs/development/testing/results/](#).
- ☐ Attach changed file list.

E. Sign-off

- Implementation Owner: _____
- Reviewer: _____
- Date: _____
- ☐ Pass [] Pass with Exceptions [] Fail
- Notes:
 - _____
 - _____